

CS II Review

COSC 2230 (CS III) Fall 2026 | Augustana U.

Table of Contents

- [1. Preamble: four illusions of the machine \("abstractions"\)](#)
- [2. What's object-oriented programming \(OOP\) about?](#)
- [3. Reference data types, wrappers, and primitive types](#)
- [4. Code along exercises](#)
- [5. Code along example: a fancy "hello world"](#)
- [6. PRACTICE Commented Greeting class](#)
- [7. Java vs C++ vs \(modern\) Python](#)
- [8. PRACTICE Extended Greeting](#)
- [9. Interfaces and polymorphism](#)

1. Preamble: four illusions of the machine ("abstractions")

Abstraction	Illusion
Comp arch (I/O)	"The machine is one"
Operating system	"The machine is all yours"
Application (OOP)	"The app is easy to use"
User (you)	"You're the only one for me"

- For each of these, there is an underlying truth

Abstraction	Illusion	Truth
Computer architecture	"The machine is one"	CPU - RAM - NVM
Operating system	"The machine is all yours"	multi-process management
Application (OOP)	"The app is easy to use"	interface vs. implementation
User (you)	"You're the only one for me"	multi-user management

2. What's object-oriented programming (OOP) about?

- Practically, for you right now:
 - You can define your own data type with `class`
 - You can create instances with `new` and call them with dot-notation
 - You can use library types the same way (e.g. `String`)
 - You can compose your own types into a bigger program
 - You can hide implementation and hand clients an interface (API)
- Technically, as a programming paradigm:

- A way of organizing a program around objects
 - Bundling data and operations on that data instead of procedures that pass passive data structures around
 - An object owns some state and is responsible for keeping it consistent
 - Object exposes a contract (interface) and hides its representation (encapsulation)
 - You get work done by sending messages & which code runs can depend on the object's type
 - The object's type is decided at run time (polymorphism/dynamic dispatch)
 - Types relate by inheritance/subtyping so one piece of client code works with many kinds of object
- Technically, in Java:
 - class is a blueprint; new builds an object on the heap, used via dot-notation
 - State lives in instance fields, behavior in instance methods; this is the current object
 - A constructor (class name, no return type) sets up the object's initial state
 - Encapsulation is compiler-enforced: private, package-private (default), public
 - Every class implicitly extends Object and may implement an interface (a bodiless contract)
 - Which implementation runs is picked at run time from the object's real class (dynamic dispatch)
 - Non-primitive types are reference types; the eight primitives aren't objects (wrapper classes + autoboxing bridge them)
 - final means assign-once; static means belongs to the class, not an instance
 - Objects are freed automatically (garbage collection); classes group into packages and ship as jar files

We'll look at each of these as part of a simple and an extended example, followed by in-class practice exercises and a home assignment to test your understanding (or your memory).

3. Reference data types, wrappers, and primitive types

- What is a **reference [data] type**?

A reference data type is a data type that consists of **data**, **constructors**, and **methods**. The constructor is used to create **objects** of the data type; the **data** signify the **state** of the object, and the **methods** allow to change the object's **behavior**. They are called **reference** types because a variable of that type doesn't hold the object itself - it holds a **reference** to it.

In terms of **memory**: when you create (or **instantiate**) an object of a reference type, the machine keeps track of the object using a reference, a link, or an address to a region of memory. **An object is a state machine.**

Integers (int) are also data types, but they are primitive, non-reference data types. They aren't objects at all.

- What about other languages?
 - Primitive vs. reference type is Java (and C#) specific
 - In Python, Javascript, Ruby everything is a reference
 - C/C++ have aliases (int&) that are called references
- What's a **wrapper class**?

Every primitive type has a matching **wrapper** class (int -> Integer, double -> Double, and so on), and a wrapper **is** a real reference type - it has a constructor, it lives on the heap, and a variable of that type holds a reference to it, not the value itself.

Java blurs the line on purpose: **autoboxing** automatically converts a primitive into its wrapper wherever a reference type is expected (Integer i = 5;), and **unboxing** converts back (int j = i;). You'll meet this soon: Chapter 4's generic collections, like Stack<Integer>, can only ever hold reference types - Stack<int> doesn't compile, Stack<Integer> does - and autoboxing is what makes stack.push(5) work anyway, with no visible conversion in your code.

Integer.parseInt(String) is different: it is an explicit **method call** that parses text into a primitive int. Autoboxing is an **implicit** conversion the compiler inserts for you, primitive to wrapper, with no method call written anywhere (Integer i = 5;).

4. Code along exercises

- To code along use whatever editor/IDE you normally use.
- If you don't have an editor/IDE, either try Emacs (my mojo), or, for quick access, go to Onecompiler.com/java - open, free, no ads.
- There is (usually) at least one code along exercise in every class that you need to upload to qualify as participating in the session.

5. Code along example: a fancy "hello world"

- Code along example: a minimal greeting reference type

```
public class Greeting
{
    String greeting;
    Greeting(String s) { greeting = s; }
    String getGreeting() { return greeting; }

    public static void main(String[] args)
    {
        Greeting greet = new Greeting("hello world");
        System.out.println(greet.getGreeting());
    }
}
```

6. PRACTICE Commented Greeting class

- Can you identify all the parts of this class? Enter comments for every line of the program explaining both syntax and semantics.
- Complete the assignment in class and then upload your .java file to Canvas.

```
// Name of a public class with state (data) and behavior (methods)
public class Greeting
{
    // A String type variable that contains the instance state
    // `greeting` is a reference to a String object
    String greeting;
```

```

// A constructor to create a Greeting object with a
// String type argument that becomes the greeting reference
// This removes the implicit no-arg default constructor
Greeting(String s) { greeting = s; }

// A Greeting accessor/getter method that returns the
// state of the Greeting object (a kind of behavior)
String getGreeting() { return greeting; }

// A quick self-test embedded in the class, and the program
// entry point that accepts command-line arguments (String type array)
// method signature is fixed
public static void main(String[] args)
{
    // An instance Greeting object with an initial String value
    // `new` allocates object on the heap and runs constructor
    // `greet` is an object reference, not the object itself
    Greeting greet = new Greeting("hello world");
    // Print the state of the Greeting object:
    // `println` is an instance method on a `PrintStream` object
    // prints the argument then the platform line separator
    System.out.println(greet.getGreeting());
}
}

```

- Why is Greeting not static and not void?

A class declaration is not callable so static and void have nothing to attach to. Only methods have return types. static means "belongs to the enclosing class, not an instance (waiting to be constructed)." But a top-level class has no enclosing class.

- What is this?

this is a reference to the object the constructor (or method) is currently running on:
 this.greeting means "the greeting field of this object. You don't need it here but you need it in this example: inside the constructor, greeting refers to the parameter and you have to qualify it with this to reach the field of the same name.

```
Greeting(String greeting) { this.greeting = greeting; }
```

- Is there any point to the class Greeting?

It shows the separation of **interface** vs. **implementation** that can only be achieved by defining your own reference types. It also shows how data/state can be separated from methods/behavior. Lastly, it demonstrates how a class can be tested.

- Is there anything that Greeting is missing?

What it does not show is the ability to **encapsulate** data by labelling them as private (visible only within the class), or public, or marking them final (unchangeable). Currently, you could also simply change greeting with: greet.greeting = "hello again"; and print it with System.out.println(greet.greeting);, because greeting is package-private only.

- How could Greeting be changed?

To incorporate **encapsulation**, you have to change greeting to be private and test access with a main program from outside the class (or the package, since Greeting sits in

Greeting.java).

We first separate the main client, GreetingTest.java, from the class definition, and change greeting to private to see what has changed.

We're working on the command-line to remind ourselves of the use of the Java compiler javac and the run-time engine java.

- Change your program now:
 1. Change the greeting field to private.
 2. Save the class declaration to Greeting.java
 3. Save the main test client to GreetingTest.java
 4. Test the client on the command-line
- Solution:
 1. public class Greeting becomes Greeting.java.
 2. public class GreetingTest becomes GreetingTest.java.
 3. Greeting.java and GreetingTest.java are compiled with javac.
 4. GreetingTest is run with java (it contains the main method).

```
public class Greeting
{
    private String greeting;
    public Greeting(String s) { greeting = s; }
    public String getGreeting() { return greeting; }
}
```

```
public class GreetingTest
{
    public static void main(String[] args)
    {
        Greeting greet = new Greeting("hello world");
        System.out.println(greet.getGreeting());
        // If `greeting` is `private`, this fails at compile-time:
        //greet.greeting = "hello again";
        //System.out.println(greet.greeting);
    }
}
```

Test it on the command-line:

```
cd ../src # change to directory ./src
rm -rf *.class # remove stale compiled .class files
javac Greeting.java GreetingTest.java 2>&1 # Greeting.class/GreetingTest.class
java GreetingTest # run GreetingTest.class with the JVM
```

```
error: file not found: Greeting.java
Usage: javac <options> <source files>
use --help for a list of possible options
```

7. Java vs C++ vs (modern) Python

Greeting in C++

- C++ is an object-oriented superset of the C language. It is arguably more popular and used more nowadays than Java. C++ compiles to a standalone native executable without any runtime dependencies: self-contained and shippable.
- The Greeting class in C++ is very similar to Java.
 - The I/O and string libraries have to be loaded
 - C++ has sections for access (public and private)
 - The constructor can be initialized with a list
 - Value instead of reference semantics: the object is created directly on the stack, without new.
 - No garbage collector: Greeting is only destroyed automatically when main returns. If you new it, you'd have to delete it.
- The code:

```
#include <iostream>
#include <string>

class Greeting
{
private:
    std::string greeting;

public:
    Greeting(std::string s) : greeting(s) {}
    std::string getGreeting() { return greeting; }

};

int main()
{
    Greeting greet("hello world");
    std::cout << greet.getGreeting() << std::endl;
    return 0;
}
```

hello world

hello world

Greeting in Python

- Python is the most popular high level language. It is written in C, and it is an interpreted rather than compiled language - that means that the user does not need to translate the code to machine code. Under the hood, CPython compiles the source to bytecode, which is then interpreted by a virtual machine (like Java, except that there is no compile-time type checking). There are no separate build artifacts (like .class or .jar). Much slower than both Java and C++.
- The code looks very different to C++ or Java:
 - No access modifiers like public or private (that is enforced)
 - Python is dynamically (not statically) typed: object data type is inferred and not explicitly stated.
 - No field declarations up front (like String greeting): __init__ creates self.greeting immediately, there is no static type checker enforcing that every instance has this attribute.
 - self is explicit everywhere. Java's equivalent (this) is implicit unless needed. self is required as first parameter of every instance method and used to access fields.

- main is just a function, not special as unique entry point.
- No System.out.println, just print. str() is called implicitly similar to how Java's println calls toString().

- Code:

```
class Greeting:
    def __init__(self, s):
        self.greeting = s

    def get_greeting(self):
        return self.greeting

def main():
    greet = Greeting("hello world")
    print(greet.get_greeting())

if __name__ == "__main__":
    main()
```

hello world

- In modern Python, a @dataclass decorator auto-generates system functions like __init__. This is close to Java's record (a Java 16+ feature).

```
from dataclasses import dataclass

@dataclass
class Greeting:
    greeting: str

    def get_greeting(self):
        return self.greeting

def main():
    greet = Greeting("hello world")
    print(greet.get_greeting())

if __name__ == "__main__":
    main()
```

hello world

8. PRACTICE Extended Greeting

Time = 15 min

1. Write a method setGreeting that sets the greeting value.
2. Write a class GreetingTest2 that tests the new method.
3. Set the greeting to "G'day mate" as command-line input.
4. Upload your

5. When asked, share your screen to show and explain your results.

Solution:

1. Write a method that sets the greeting value.

```
public class Greeting2
{
    private String greeting;
    public Greeting2(String s) { greeting = s; }
    public String getGreeting() { return greeting; }
    public void setGreeting(String s) { greeting = s; }
}
```

2. Write a test in the testing class for the method.

```
public class GreetingTest2
{
    public static void main(String[] args)
    {
        Greeting2 greet = new Greeting2("hello world");
        System.out.println(greet.getGreeting());
        greet.setGreeting("G'day mate");
        System.out.println(greet.getGreeting());
    }
}
```

```
cd ../src
rm -rf *.class
javac Greeting2.java GreetingTest2.java
java GreetingTest2
```

```
hello world
G'day mate
```

3. Set the greeting to "G'day mate" as command-line input.

```
public class GreetingTest3
{
    public static void main(String[] args)
    {
        Greeting2 greet = new Greeting2("hello world");
        System.out.println(greet.getGreeting());
        greet.setGreeting(args[0]);
        System.out.println(greet.getGreeting());
    }
}
```

```
cd src
rm -rf *.class
javac Greeting2.java GreetingTest3.java
java GreetingTest3 "G'day, mate"
```

```
hello world
G'day, mate
```

9. Interfaces and polymorphism

- What does **interface vs. implementation** actually look like in code - not just as a design principle, but as a language mechanism?

A Java interface declares a contract - method signatures with no bodies - that any class can promise to fulfill with implements. Once a class implements an interface, you can hold a reference to one of its objects using the **interface** type instead of the concrete class type. Which method body actually runs is decided at **run-time**, based on the real object's class, not the declared type of the variable holding it. This mechanism is called **polymorphism** (or **dynamic dispatch**).

- Why does this matter for what's coming?

Stack and Queue classes are built on exactly this mechanism. Both implement Java's built-in `Iterable<Item>` interface, which requires exactly one method, `iterator()`, returning an object that itself implements `Iterator<Item>` (`hasNext()` and `next()`). That one small interface contract is what lets you write `for (String s : stack)` instead of manually indexing into the stack's internals - the `for-each` loop works by calling `iterator()`, `hasNext()`, and `next()` through the interface, without knowing or caring whether `Stack` is actually implemented with an array or a linked list underneath. Same idea as `Greeter` below - just applied to something you'll be building yourself soon.

- A minimal example: one interface, two unrelated implementations.
- First, the contract itself - just a name and a method **signature**, no body at all. This compiles on its own but does nothing yet; it only says "anything that calls itself a `Greeter` must have a `sayHello()` method that returns a `String`."

```
public interface Greeter
{
    String sayHello();
}
```

- Now the first class that **keeps** that promise. Note `implements Greeter` in the class header - that's the only thing linking this class to the interface above.

```
public class FormalGreeter implements Greeter
{
    public String sayHello() { return "Good day to you."; }
}
```

- A second class, also implementing `Greeter`, with a completely different `sayHello()` body. It has no relationship to `FormalGreeter` at all - they only share the same contract.

```
public class CasualGreeter implements Greeter
{
    public String sayHello() { return "Hey, what's up?"; }
}
```

- Finally, the client. This is where dynamic dispatch actually happens: the array is declared as Greeter[], not as any concrete class, and the loop calls sayHello() through that interface type - watch for which message prints for which object.

```
public class GreeterTest
{
    public static void main(String[] args)
    {
        Greeter[] greeters = { new FormalGreeter(), new CasualGreeter() };
        for (Greeter g : greeters)
            System.out.println(g.sayHello());
    }
}
```

Compile and run on the command line

```
cd ../src
rm -f *.class
javac Greeter.java FormalGreeter.java CasualGreeter.java GreeterTest.java
java GreeterTest
```

```
Good day to you.
Hey, what's up?
```

- Neither FormalGreeter nor CasualGreeter knows the other exists - the only thing tying them together is the shared Greeter contract. The loop in GreeterTest never mentions either concrete class by name; g.sayHello() calls whichever implementation the **actual** object in that array slot has, decided at run-time. That's the payoff: code written once against the **interface** keeps working unchanged if you add a third, fourth, tenth Greeter later.
- A note on **abstract classes**: Java has a second mechanism for this same idea, abstract class - a class that, like an interface, can declare methods with no body, but, unlike an interface, can also hold state and provide ready-made methods for subclasses to inherit.

This course's textbook **doesn't use them anywhere** - every polymorphism example in Sedgewick & Wayne is built on **interfaces**. If you see abstract class elsewhere - other courses, other code - it's solving this same problem with a different, heavier tool, not a concept you're missing from here.

Author: Marcus Birkenkrahe

Created: 2026-09-04 Fri 20:18